

Question Bank — Week 8

Searching (Lecture 08) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Questions: *week08-questions.md*. Commit to your own answer before reading one here.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	a	M07	b	M13	a	M19	c
M02	c	M08	d	M14	d	M20	b
M03	b	M09	a	M15	c	M21	d
M04	d	M10	c	M16	b	M22	a
M05	a	M11	d	M17	d		
M06	c	M12	b	M18	a		

W8-M01 — a. The other three all rely on the order to skip elements.

W8-M02 — c. $\lfloor \log_2 1000 \rfloor + 1 = 9 + 1 = 10$. 1,000 (a) is linear search's worst case.

W8-M03 — b. $\lfloor \log_2 10^6 \rfloor + 1 = 19 + 1 = 20$.

W8-M04 — d. Nothing checks the order — checking would cost $O(n)$. The comparison with the middle then discards the wrong half, and a value that is present can be reported as -1 .

W8-M05 — a. $\text{mid} = (\text{lo} + 15) // 2 = 7, V[7] = 24$.

W8-M06 — c. $24 > 20$, so $\text{hi} = 6$; $\text{mid} = (\text{lo} + 6) // 2 = 3, V[3] = 12$.

W8-M07 — b. It is what makes “the range is empty” mean “the target is not there”.

W8-M08 — d. With $\text{lo} = \text{hi} = 0$ the loop never runs, so the only candidate is never compared (W8-B1).

W8-M09 — a. Joshua Bloch's 2006 bug report: arrays of more than about 2^{30} elements. Fix: $\text{lo} + (\text{hi} - \text{lo}) / 2$. Python integers do not overflow.

W8-M10 — c. The slices copy $n/2 + n/4 + \dots < n$ elements in total (W8-K2).

W8-M11 — d. One frame per halving: about $\log_2 n$ frames. The loop form is $O(1)$ (a).

W8-M12 — b. The first index with $v[i] \geq 2$.

W8-M13 — a. The first index with $v[i] > 2$: the 5 at index 4.

W8-M14 — d. No element is ≥ 8 , so the answer is $\text{len}(v) = 6$ — the insertion point at the end. The bounds never return -1 (a).

W8-M15 — c. $\text{upper_bound} = 5$ and $\text{lower_bound} = 1$: $5 - 1 = 4$.

W8-M16 — b. And bisect_right is upper_bound .

W8-M17 — d. $\sqrt{100} = 10$: at most 10 jumps and 10 steps inside a block.

W8-M18 — a. $n/m + m$ is smallest at $m = \sqrt{n}$, giving $2\sqrt{n}$. Jump search is $O(\sqrt{n})$, not $O(\log n)$ (d).

W8-M19 — c. It checks slot 0, then doubles: $v[16]$ is the first bound not smaller than the target, and binary search runs on slots 8 to 16.

W8-M20 — b. Doubling needs no end, and it costs $O(\log i)$ for a target at index i .

W8-M21 — d. One huge value makes every guess land at the left end, and the range shrinks by one per step (W8-T5).

W8-M22 — a. $0 + (70 - 10)(9 - 0)/(100 - 10) = 540/90 = 6$, and $v[6] = 70$: found in one probe.

Part B — Short answer and essay

W8-E1 (4)

- **Algorithm:** keep $\text{lo} = 0$, $\text{hi} = n - 1$; while $\text{lo} \leq \text{hi}$, compare the target with $\text{values}[\text{mid}]$, $\text{mid} = (\text{lo} + \text{hi}) // 2$: equal $\rightarrow$ return mid ; smaller middle $\rightarrow \text{lo} = \text{mid} + 1$; larger $\rightarrow \text{hi} = \text{mid} - 1$. Return -1 when the range is empty.
- **Invariant:** if the target is in the list, it is in $\text{values}[\text{lo}..\text{hi}]$. True at the start (whole list); each step removes only elements that the sorted order proves cannot be the target; so when the range is empty, the target is absent — the -1 is correct.
- **Count:** each unsuccessful comparison leaves at most half the range; after $\lfloor \log_2 n \rfloor$ halvings one element is left, and one more comparison settles it: $O(\log n)$.
- **Caller's guarantee:** the list is **sorted** (a precondition — not checked).

W8-E2 (4)

- while $\text{lo} < \text{hi}$ (inclusive hi): the last candidate is never compared — $[42]$ searched for 42 gives -1 . **Fix:** $\text{lo} \leq \text{hi}$.
- $\text{hi} = \text{mid}$: when the range is one element and it is too big, nothing changes — $[1, 3]$ searched for 0 loops for ever. **Fix:** $\text{hi} = \text{mid} - 1$.
- $\text{lo} = \text{mid}$: with $\text{lo} = 0$, $\text{hi} = 1$ and $\text{values}[0]$ too small, mid stays 0 — loops for ever. **Fix:** $\text{lo} = \text{mid} + 1$.
- $\text{mid} = (\text{lo} + \text{hi}) / 2$ in Java/C: overflows on arrays over about 2^{30} elements, giving a negative index. **Fix:** $\text{lo} + (\text{hi} - \text{lo}) / 2$.
- (Also accepted: recursing on slices — returns an index into the slice, W8-B3.)

W8-E3 (4)

- `lower_bound(v, x)`: the first index i with $v[i] \geq x$; `upper_bound(v, x)`: the first with $v[i] > x$. Both in $O(\log n)$, on a sorted list.
- **Uses**: the insertion point that keeps the list sorted (`lower_bound`); the number of copies of x (`upper_bound - lower_bound`); the first occurrence (`lower_bound`, then check $v[i] == x$); the count of values in $[a, b)$.
- **Never – 1**: they answer “where”, not “whether” — every x has a position in the order, from 0 (before everything) to $\text{len}(v)$ (after everything). That is why the search range is half-open, $[0, \text{len}(v))$: $\text{len}(v)$ must be a possible answer.
- Python: `bisect_left` and `bisect_right`.

W8-E4 (3)

- **Jump**, $O(\sqrt{n})$: moves only forward, in blocks of $\sqrt{n}$ — for storage where going back is expensive (tape, a stream read in order).
- **Exponential**, $O(\log i)$ for a target at index i : doubles a bound, then binary searches — for sequences of unknown length, or targets near the front.
- **Interpolation**, $O(\log \log n)$ on average for evenly spread values but $O(n)$ in the worst case: guesses the position from the values — for large, uniformly distributed keys (numeric IDs, timestamps) where you can trust the distribution.

W8-E5 (3)

- Linear search k times: $O(kn)$. Sort once and binary search k times: $O(n \log n + k \log n)$.
- Sorting pays once $kn > n \log n$, i.e. once **k exceeds about $\log_2 n$** .
- **Example**: $n = 1,000,000$. One linear search: up to 10^6 reads. Sorting: about $n \log_2 n \approx 2 \times 10^7$ comparisons, then 20 per search. After about 20 searches the sort has paid for itself.
- If the data also changes often, a sorted array’s $O(n)$ insertion is the catch: a balanced search tree (Week 11) or a hash table (Week 13) may fit better.

Part C — Trace the algorithms**W8-T1**

Target	lo	hi	mid	values[mid]	Decision
23	0	9	4	16	$16 < 23 \rightarrow \text{lo} = 5$
	5	9	7	56	$56 > 23 \rightarrow \text{hi} = 6$
	5	6	5	23	found: 5
60	0	9	4	16	$16 < 60 \rightarrow \text{lo} = 5$
	5	9	7	56	$56 < 60 \rightarrow \text{lo} = 8$
	8	9	8	72	$72 > 60 \rightarrow \text{hi} = 7$
	8	7			$\text{lo} > \text{hi}$: -1

For 60 the loop ends with $lo = 8$: the index where 60 **would be inserted** to keep the list sorted (between 56 and 72) — exactly `lower_bound(values, 60)`.

W8-T2

Function	lo	hi	mid	v[mid]	Direction
<code>lower_bound(v, 2)</code>	0	6	3	2	$2 < 2$? no $\rightarrow$ left: $hi = 3$
	0	3	1	2	no $\rightarrow$ left: $hi = 1$
	0	1	0	1	$1 < 2 \rightarrow$ right: $lo = 1$
	1	1			empty: return 1
<code>upper_bound(v, 2)</code>	0	6	3	2	$2 \leq 2 \rightarrow$ right: $lo = 4$
	4	6	5	7	$7 \leq 2$? no $\rightarrow$ left: $hi = 5$
	4	5	4	5	no $\rightarrow$ left: $hi = 4$
	4	4			empty: return 4

The two differ only in the comparison that sends the search right: $<$ for `lower_bound`, $\leq$ for `upper_bound`. Neither stops when it meets a 2.

W8-T3

```

binary_search_recursive(V, 45)
  lo=0 hi=15 mid=7 V[7]=24 < 45
    lo=8 hi=15 mid=11 V[11]=38 < 45
      lo=12 hi=15 mid=13 V[13]=45: return 13      (3 calls)

binary_search_recursive(V, 7)
  lo=0 hi=15 mid=7 V[7]=24 > 7
    lo=0 hi=6 mid=3 V[3]=12 > 7
      lo=0 hi=2 mid=1 V[1]=6 < 7
        lo=2 hi=2 mid=2 V[2]=8 > 7
          lo=2 hi=1: empty, return -1      (5 calls)

```

The search for 7 is 5 frames deep — about $\log_2 16 + 1$. Each call returns the value of the call it made, unchanged, so the -1 or the index travels back up the stack.

W8-T4. Step $\sqrt{25} = 5$. The jumps read the last element of each block: $v[4] = 9$, $v[9] = 19$, $v[14] = 29$ — all smaller than 30 — then $v[19] = 39$, not smaller: stop. The walk starts at index 15: $v[15] = 31$ is already larger than 30, so 30 is not there. **Returns -1** after reading 5 elements.

W8-T5

Search	lo	hi	pos	v[pos]	Result
70 in [10, ..., 100]	0	9	$0 + 60 \cdot 9 // 90 = 6$	70	found, 1 probe
75 in [10, ..., 100]	0	9	$0 + 65 \cdot 9 // 90 = 6$	70	$70 < 75 \rightarrow lo = 7$
	7	9			$75 < v[7] = 80$: outside the range $\rightarrow$ -1 , 1 probe

Search	lo	hi	pos	v[pos]	Result
9 in [1, ..., 9, 1000]	0	9	$0 + 8 \cdot 9 // 999 = 0$	1	lo = 1
	1	9	$1 + 7 \cdot 8 // 998 = 1$	2	lo = 2
	...		each guess is lo		...
	8	9	$8 + 0 = 8$	9	found, 9 probes

On evenly spread values the guess is exact. With one huge value, the straight line from $v[lo]$ to 1000 puts every target at the far left, so the range shrinks by **one** element per probe: interpolation search has become a linear search — its $O(n)$ worst case. Binary search would need at most 4 probes here.

Part D — Complexity analysis

W8-K1 — $\Theta(m \log n)$. m searches of $O(\log n)$ each (worst case). With `linear_search`: $\Theta(mn)$. **If a were unsorted**, binary search would give wrong answers; sort it first: $O(n \log n + m \log n)$ — worth it once m exceeds about $\log n$ (W8-E5).

W8-K2 — $\Theta(n)$. The slice copies about half the list, so $T(n) = T(n/2) + \Theta(n)$. Unrolled: $n/2 + n/4 + \dots + 1 < n - \Theta(n)$, no better than linear search. (And the index it returns is wrong — W8-B3.)

W8-K3 — $\Theta(\log n + c)$. The binary search is $O(\log n)$; the walk visits all c copies. **Worst case**: a list that is all $x - \Theta(n)$. **In** $O(\log n)$: `upper_bound(v, x) - lower_bound(v, x)`, two binary searches that never walk.

Part E — Find and fix the bug

W8-B1. `while lo < hi` with an inclusive `hi` stops while one candidate, `values[lo]`, is still unchecked. `binary_search([42], 42)` returns `-1`, and so do `binary_search([1, 3], 3)` and `binary_search([1, 3, 5, 7, 9], 9)` — the tests for one element and for the last element fail. **Fix**: `while lo <= hi`.

W8-B2. `hi = mid` does not move past `mid`, which has just been ruled out. When the range is a single too-large element, `lo = hi = mid` and nothing changes: `binary_search([1, 3], 0)` and `binary_search([1, 3, 5], 4)` loop for ever. (It still finds values that are present, which is why it can survive a few tests.) **Fix**: `hi = mid - 1`.

W8-B3. Two bugs. The index returned from a call on `values[mid + 1:]` is an index into the **slice**: on `[1, 3, 5, 7, 9]` it returns 0 for 7 and 1 for 9 (right answers 3 and 4), while 1, 3 and 5 happen to come out right. And every slice copies: $\Theta(n)$ in total (W8-K2). **Fix**: pass `lo` and `hi` and index the original list — or add `mid + 1` to a non-negative result from the right half, which fixes the index but not the cost.

W8-B4. When `values[hi] == values[lo]` — for example `[5, 5, 5]`, or any range that has shrunk to equal values — the division is by zero: `ZeroDivisionError`. **Fix:** before computing `pos`, if `values[hi] == values[lo]`, return `lo` if it equals the target, else `-1`.

Part F — Write the code

W8-C1

```
def search_rotated(values, target):
    lo, hi = 0, len(values) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        value = values[mid]
        if value == target:
            return mid
        first = values[lo]
        if first <= value:                # left half lo..mid is sorted
            if first <= target < value:
                hi = mid - 1
            else:
                lo = mid + 1
        else:                            # right half mid..hi is sorted
            if value < target <= values[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

Why one half is sorted: the rotation creates a single “drop” (from the largest value to the smallest). Cutting at `mid` puts that drop in at most one of the two halves, so the other is in order. For the sorted half, two comparisons tell whether the target lies inside it; if not, it can only be in the other half. Still one halving per step: $O(\log n)$.

W8-C2

```
def integer_sqrt(n):
    if n < 0:
        raise ValueError("no integer square root of a negative number")
    lo, hi = 0, n                        # the answer is in 0..n
    while lo < hi:
        mid = (lo + hi + 1) // 2         # round up, or lo = mid never moves
        if mid * mid <= n:
            lo = mid                     # mid is possible; the answer is >= mid
        else:
            hi = mid - 1
```

```
return lo
```

The test “ $r^2 \leq n$ ” is true for every r up to the answer and false after it, and that is all binary search needs — a **monotonic** yes/no question, not a sorted list. Note the $+ 1$ in mid : with $\text{lo} = \text{mid}$ as an update, rounding down would loop for ever on $\text{lo} = 0$, $\text{hi} = 1$ (the bug of W8-E2). $O(\log n)$ multiplications; for $n = 10^{18}$ about 60.

W8-C3

```
def find_peak(values):
    lo, hi = 0, len(values) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if values[mid] < values[mid + 1]:      # rising: a peak lies to the right
            lo = mid + 1
        else:                                  # falling or flat: mid or left
            hi = mid
    return lo
```

Why half can go: if $\text{values}[\text{mid}] < \text{values}[\text{mid} + 1]$, walk right from $\text{mid} + 1$: the values either keep rising to the end — then the last element is a peak, because the outside counts as $-\infty$ — or they fall somewhere, and the top before the fall is a peak. Either way a peak lies in $\text{mid} + 1..hi$. The same argument the other way keeps $\text{lo}..mid$. The list is not sorted, but the question “is there a peak on this side?” still has a guaranteed answer, and that is enough to halve.

W8-C4

```
def closest_value(values, target):
    lo, hi = 0, len(values)                # lower_bound, half-open
    while lo < hi:
        mid = (lo + hi) // 2
        if values[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    if lo == len(values):
        return values[lo - 1]
    if lo == 0:
        return values[0]
    before, after = values[lo - 1], values[lo]
    return before if target - before <= after - target else after
```

lo is where the target would be inserted: every value before it is smaller, every value from it on is at least as large. So the closest value is one of the two neighbours of that position — or the only one, at either end. $\leq$ breaks a tie toward the smaller value.

W8-C5

```

def min_capacity(weights, days):
    def fits(capacity):
        needed, load = 1, 0
        for w in weights:
            if load + w > capacity:
                needed += 1
                load = 0
            load += w
        return needed <= days

    lo, hi = max(weights), sum(weights)
    while lo < hi:
        mid = (lo + hi) // 2
        if fits(mid):
            hi = mid                # mid works; maybe smaller does too
        else:
            lo = mid + 1
    return lo

```

Why binary search: if a capacity works, every larger capacity works too — the answer to “does capacity c fit?” is *no, no, ..., no, yes, yes, ...*. The smallest “yes” is a lower_bound on that yes/no sequence, even though the sequence is never stored. The capacity lies between $\max(\text{weights})$ (the heaviest package must fit) and $\text{sum}(\text{weights})$ (everything in one day). Each check is one greedy pass, $O(n)$, and there are $O(\log S)$ checks: $O(n \log S)$. Trying every capacity from the maximum upward would take $O(nS)$ — hopeless in the test, where S is 2×10^{10} .